Qus1: Identify and obtain a suitable real-world dataset related to sales.

```
# If kagglehub is not installed
!pip install kagglehub
```

```
Requirement already satisfied: kagglehub in /usr/local/lib/python3.12/dist-packages (0.3.13)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kagglehub) (26.0)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from kagglehub) (6.0.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kagglehub) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kagglehub) (4.67.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->kagg
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kagg
```

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("brijbhushannanda1979/bigmart-sales-data")

print("Path to dataset files:", path)
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed: 0.3.13), please consider upgrading t
Downloading from https://www.kaggle.com/api/v1/datasets/download/brijbhushannanda1979/bigmart-sales-data?dataset_
100%|██████████| 307k/307k [00:00<00:00, 658kB/s]Extracting files...
Path to dataset files: /root/.cache/kagglehub/datasets/brijbhushannanda1979/bigmart-sales-data/versions/1
```

```
import kagglehub
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")
```

```
import kagglehub

# Download dataset using kagglehub
# The dataset was previously downloaded to the following path.
# We will explicitly set the path to this location to ensure correctness.
path = '/root/.cache/kagglehub/datasets/brijbhushannanda1979/bigmart-sales-data/version

print("Dataset path:", path)
```

```
Dataset path: /root/.cache/kagglehub/datasets/brijbhushannanda1979/bigmart-sales-data/versions/1
```

```
import os

print("Dataset path:", path)
print("\nFiles inside dataset folder:")
os.listdir(path)
```

```
Dataset path: /root/.cache/kagglehub/datasets/brijbhushannanda1979/bigmart-sales-data/versions/1

Files inside dataset folder:
['Test.csv', 'Train.csv']
```

```
import pandas as pd
import os

# Load training data
df = pd.read_csv(os.path.join(path, "Train.csv"))

# Preview
df.head()
```

	Item_Identifier	Item_Weight	Item_Fat_Content	Item_Visibility	Item_Type	Item_MRP	Outlet_Identifier	Outlet_
0	FDA15	9.30	Low Fat	0.016047	Dairy	249.8092	OUT049	
1	DRC01	5.92	Regular	0.019278	Soft Drinks	48.2692	OUT018	
2	FDN15	17.50	Low Fat	0.016760	Meat	141.6180	OUT049	
3	FDX07	19.20	Regular	0.000000	Fruits and Vegetables	182.0950	OUT010	
4	NCD19	8.93	Low Fat	0.000000	Household	53.8614	OUT013	

The BigMart Sales dataset was obtained from Kaggle. It contains 8,523 records and 12 attributes related to retail sales such as item weight, MRP, outlet type, outlet size, and total sales. The dataset represents real-world sales data used for descriptive analytics and visualization.

Qus2: Perform data preprocessing and compute descriptive statistics for at least three numerical attributes

```
#Check missing values
df.isnull().sum()
```

	0
Item_Identifier	0
Item_Weight	1463
Item_Fat_Content	0
Item_Visibility	0
Item_Type	0
Item_MRP	0
Outlet_Identifier	0
Outlet_Establishment_Year	0
Outlet_Size	2410
Outlet_Location_Type	0
Outlet_Type	0
Item_Outlet_Sales	0

dtype: int64

```
# Fill missing Item_Weight with mean
df['Item_Weight'].fillna(df['Item_Weight'].mean(), inplace=True)

# Fill missing Outlet_Size with mode
df['Outlet_Size'].fillna(df['Outlet_Size'].mode()[0], inplace=True)

# Verify
df.isnull().sum()
```

```
/tmp/ipython-input-32719364.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series t
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on wh

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'


  df['Item_Weight'].fillna(df['Item_Weight'].mean(), inplace=True)
/tmp/ipython-input-32719364.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series t
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on wh

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'


  df['Outlet_Size'].fillna(df['Outlet_Size'].mode()[0], inplace=True)
```

	0
Item_Identifier	0
Item_Weight	0
Item_Fat_Content	0
Item_Visibility	0
Item_Type	0
Item_MRP	0
Outlet_Identifier	0
Outlet_Establishment_Year	0
Outlet_Size	0
Outlet_Location_Type	0
Outlet_Type	0
Item_Outlet_Sales	0

dtype: int64

```
Q1 = df['Item_Outlet_Sales'].quantile(0.25)
Q3 = df['Item_Outlet_Sales'].quantile(0.75)
IQR = Q3 - Q1

df = df[
    (df['Item_Outlet_Sales'] >= Q1 - 1.5 * IQR) &
    (df['Item_Outlet_Sales'] <= Q3 + 1.5 * IQR)
]
```

```
df[['Item_Weight', 'Item_MRP', 'Item_Outlet_Sales']].describe()
```

	Item_Weight	Item_MRP	Item_Outlet_Sales
count	8290.000000	8290.000000	8290.000000
mean	12.855821	138.728957	2034.951441
std	4.252761	61.407454	1474.938933
min	4.555000	31.290000	33.290000
25%	9.300000	92.721850	806.450250
50%	12.857645	140.615400	1733.743200
75%	16.100000	182.986450	2972.131200
max	21.350000	266.888400	6208.585000

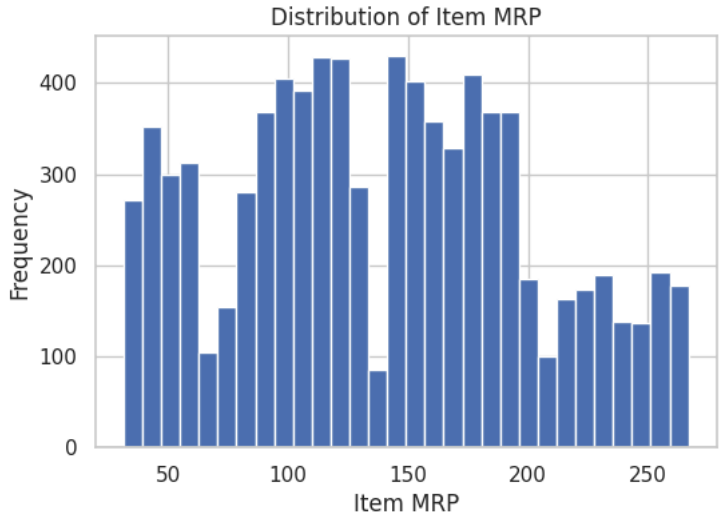
```
for col in ['Item_Weight', 'Item_MRP', 'Item_Outlet_Sales']:
    print(f"\nStatistics for {col}")
    print("Mean:", df[col].mean())
    print("Median:", df[col].median())
    print("Mode:", df[col].mode()[0])
    print("Standard Deviation:", df[col].std())
```

```
Statistics for Item_Weight
Mean: 12.855821238001345
Median: 12.857645184135976
Mode: 12.857645184135976
```

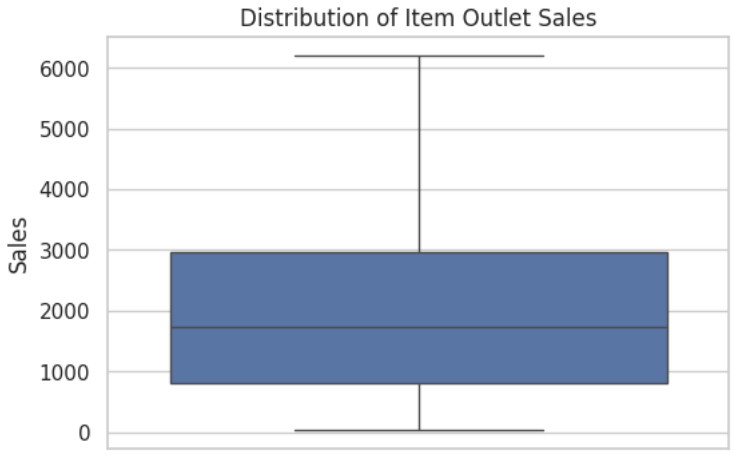
```
Standard Deviation: 4.252761369837281

Statistics for Item_MRP
Mean: 138.7289573703257
Median: 140.6154
Mode: 172.0422
Standard Deviation: 61.407453803742385

Statistics for Item_Outlet_Sales
Mean: 2034.9514412545234
Median: 1733.7432
Mode: 958.752
Standard Deviation: 1474.9389334356697
```

Ques:3 Perform univariate analysis for numerical attributes using appropriate visualizations.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Histogram for Item_MRP
plt.figure(figsize=(6,4))
plt.hist(df['Item_MRP'], bins=30)
plt.title("Distribution of Item MRP")
plt.xlabel("Item MRP")
plt.ylabel("Frequency")
plt.show()
```



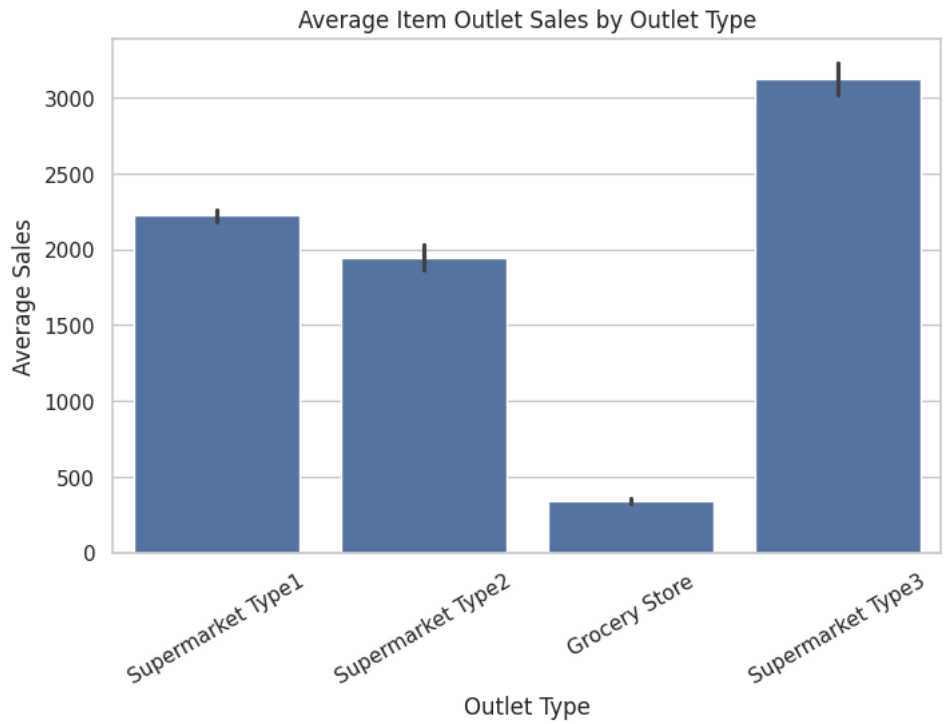

```
#univerate analysis
# Boxplot for Item_Outlet_Sales
plt.figure(figsize=(6,4))
sns.boxplot(y=df['Item_Outlet_Sales'])
plt.title("Distribution of Item Outlet Sales")
plt.ylabel("Sales")
plt.show()
```



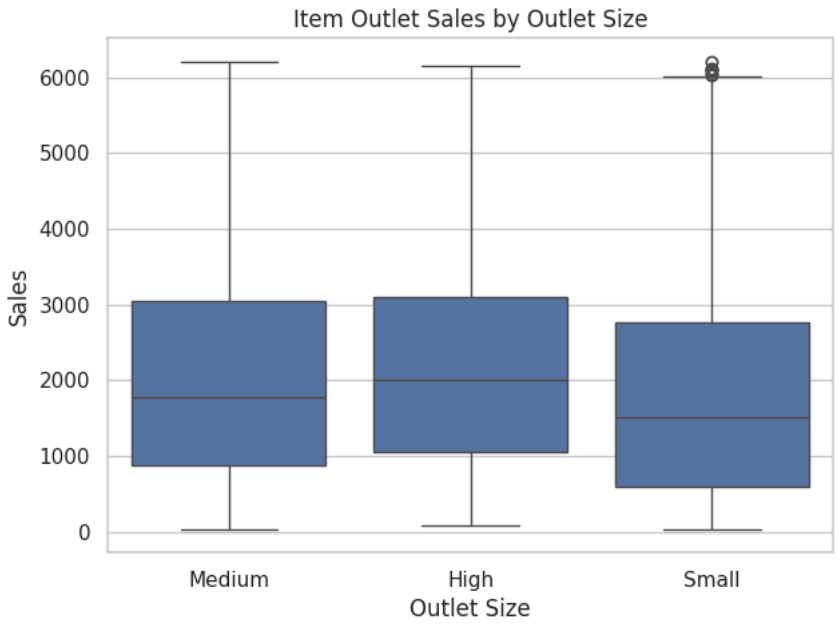
```
# Histogram for Item_Outlet_Sales
plt.figure(figsize=(6,4))
plt.hist(df['Item_Outlet_Sales'], bins=30)
plt.title("Distribution of Item Outlet Sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.show()
```

Ques:4 Perform bivariate analysis between categorical and numerical attributes using appropriate visualizations

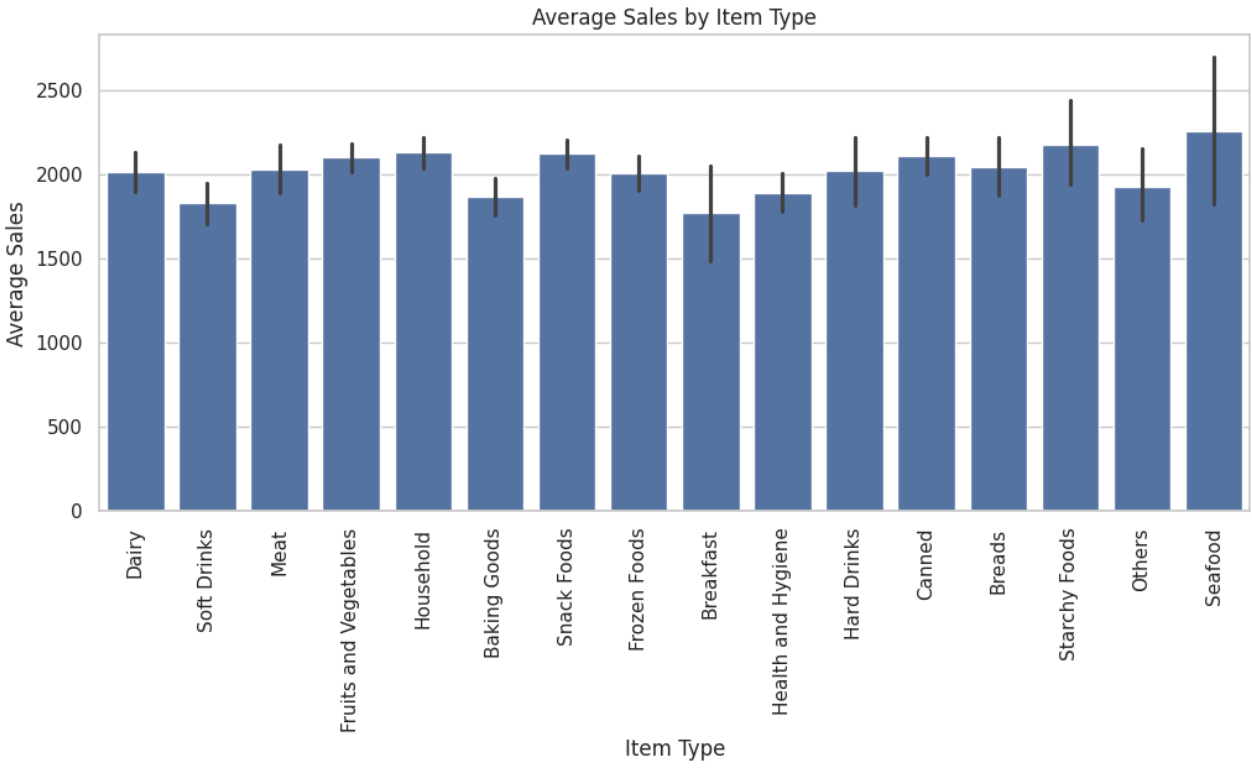
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8,5))
sns.barplot(x='Outlet_Type', y='Item_Outlet_Sales', data=df)
plt.title("Average Item Outlet Sales by Outlet Type")
plt.xlabel("Outlet Type")
plt.ylabel("Average Sales")
plt.xticks(rotation=30)
plt.show()
```




```
plt.figure(figsize=(7,5))
sns.boxplot(x='Outlet_Size', y='Item_Outlet_Sales', data=df)
plt.title("Item Outlet Sales by Outlet Size")
plt.xlabel("Outlet Size")
plt.ylabel("Sales")
plt.show()
```




```
plt.figure(figsize=(12,5))
sns.barplot(x='Item_Type', y='Item_Outlet_Sales', data=df)
plt.title("Average Sales by Item Type")
plt.xlabel("Item Type")
plt.ylabel("Average Sales")
plt.xticks(rotation=90)
plt.show()
```



Ques:5 Create a simple dashboard using multiple visualizations and derive insights from the dataset

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(14,10))

# 1. Sales Distribution
```

```
plt.subplot(2,2,1)
sns.histplot(df['Item_Outlet_Sales'], kde=True)
plt.title("Sales Distribution")

# 2. Sales by Outlet Size
plt.subplot(2,2,2)
sns.boxplot(x='Outlet_Size', y='Item_Outlet_Sales', data=df)
plt.title("Sales by Outlet Size")

# 3. Sales by Outlet Type
plt.subplot(2,2,3)
sns.barplot(x='Outlet_Type', y='Item_Outlet_Sales', data=df)
plt.xticks(rotation=30)
plt.title("Sales by Outlet Type")

# 4. MRP vs Sales
plt.subplot(2,2,4)
sns.scatterplot(x='Item_MRP', y='Item_Outlet_Sales', data=df)
plt.title("MRP vs Item Outlet Sales")

plt.tight_layout()
```